Lifelong Learning Techniques

for an Origami Robot

by

E. De Vroey

Literature Study Report

Student number: 4685156
Duration: September 1, 2023 – November 30, 2023
Supervisor: Dr. J. Kober

TU Delft

Summary

Origami robots, characterized by their construction using flexible materials and origami techniques, are becoming an attractive alternative to conventional robotics, due to their compliance and adaptability. However, these traits also lead to a faster rate of degradation than traditional robots, necessitating the development of adaptive lifelong learning strategies for their effective and sustained operation.

This literature study presents an extensive overview of lifelong learning techniques. It provides a foundation for the implementation of lifelong learning in the control of an origami robot. However, to review a wide range of possible solutions, the study does not exclusively focus on robotics applications, but rather on the broader domain of lifelong adaptation of an agent to new tasks, environments, and dynamics.

Central to the study is the exploration of the challenges inherent in lifelong learning, notably catastrophic forgetting, efficient memory handling, and the detection of shifts in the data landscape. These challenges highlight the importance of an agent's ability to retain previously learned knowledge, efficiently store and retrieve memories, and recognize significant changes in its operational environment or in its own mechanical dynamics.

The study delves into various machine-learning-based methods for managing the evolving dynamics of robots. It examines 'blackbox' methods, which, despite their lack of interpretability, are able to offer certain benefits in terms of performance for lifelong learning tasks, such as incremental updates or more robust results in uncertain environments. These methods are examined in terms of control policies and dynamic models that enable agents to adapt continually to changing conditions.

Further, the study explores dual architectures in lifelong learning, which consist of separate modules tackling different aspects of the challenge. This includes policy/control modules combined with knowledge bases, as well as multiple control policies designed to handle varying levels of uncertainty. In addition, some methods in which architectures can be expanded to obtain lifelong learning capabilities are explored. The composition and maintenance of knowledge bases are also addressed, addressing the diversity in the type and form of knowledge stored, its retrieval, and the strategies for keeping the knowledge base relevant and up to date throughout an agent's lifespan.

The study concludes by identifying a number of promising approaches and unexplored research avenues: Data-driven dynamic models are versatile for various settings, and dual architectures provide more interpretable solutions. Architectural expansions to existing models allow for lifelong learning without altering the base control method. On the other hand, the literature lacks emphasis on real-time applications and often focuses on policies handling discrete changes in tasks and environments. The study suggests exploring more flexible architectures and *one-size-fits-all* expansions for a wide range of control policies.

In summary, this literature study provides a foundational perspective on the application of lifelong learning in the field of origami robotics. It highlights the necessity and challenges of adaptive learning in this context and outlines potential machine learning and control strategies to address these challenges, laying the groundwork for further research and development in adaptive control and learning for an origami robot.

Contents

1
Introduction

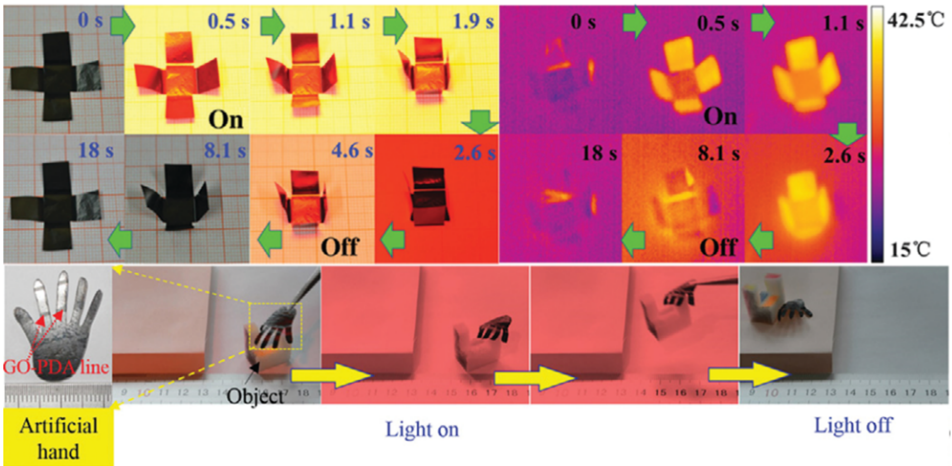
All robots suffer a certain degree of wear during their lifetime: unforeseen damages to structures, joints that lose friction, diminishing motor and battery power, etc. These evolutions change dynamics of the robot, which can be detrimental to its control. Consider a simplified example where a robot arm that normally reaches its target with a certain torque and timing might miss due to weaker motors or smoother joints. A solution to this could be that the user has instructions to update the software of the robot every few weeks; or preferably, the robot receives software updates which the user only needs to download and install; or even better, the robot automatically updates itself when necessary… Enter *lifelong learning*, where systems adapt continuously throughout their operational life.

This literature study aims to provide an extensive overview of techniques for enabling lifelong learning in robot control. The study will provide essential background for the thesis, which will aim to achieve lifelong learning in the control of a small origami robot.

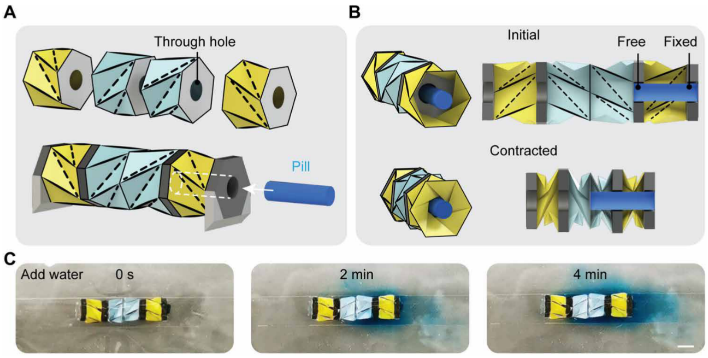
First, a note on 'origami robots': These are not robots that fold origami, but rather robots constructed by origami techniques. This includes designs made out of flexible materials such as thin plastics or paper, where the folds can provide both structural integrity and serve as actuation mechanisms. These approaches offer advantages such as compact configurations and miniaturization, self-deployment, but often also require~~and~~ unconventional actuation mechanisms (magnetic, thermal deformation, … See fig. 1.1a for a good example of an unconventional actuation method for a highly miniaturized origami robot.) (Tang & Wei, 2022). Origami robots are a subset of soft robots, a field that has significantly evolved recently. Their flexibility and non-rigid materials make them inherently compliant and adaptable, which comes with numerous advantages, for example, the use of renewable and biodegradable materials (Hartmann et al., 2021). Furthermore, soft robots are safer for human interaction, more maneuverable, and energy-efficient in locomotion due to their flexible nature (Kim et al., 2021; Sun et al., 2021). The combination of soft (biodegradable) materials, miniaturization, and innovative actuation methods has spawned many new concepts and various fields, such as search and rescue (Hawkes et al., 2017) and medical applications (Ze et al., 2022, see fig. 1.1b).

1.1. Application to Soft (Origami) Robots

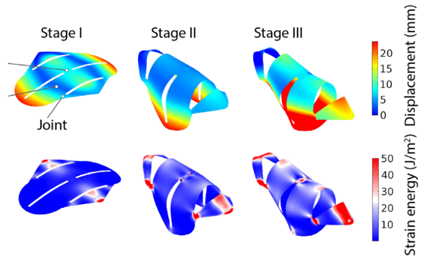
Why apply lifelong learning to soft (origami) robots? First, their flexibility makes modeling them a challenge, and their control thus tends towards machine-learning-based methods (Kim et al., 2021). Additionally, consider the example about the updates for a conventional robot: whereas manually updating a robot every few weeks might not be such an expensive trade-off, updating a soft robot every few hours is considerably more cumbersome. Due to their materials, soft



(a) *"A programmed gradient robot with the high light thermal conversion efficiency structure of the rGO layer that enables instant self-fold can achieve predesigned shapes and grasp and hold objects"* (Tang & Wei, 2022). Figure and caption obtained from Tang and Wei (2022).



(b) *"Conceptual scheme for drug storage and release using the Kresling crawler. (**A**) Exploded view and assembly of the modified Kresling crawler with a through hole. The internal cavity of the front Kresling unit is used for pill storage. (**B**) Pill positions at initial and contracted states of the Kresling crawler. The crawler contracts without interfering with the cylindrical pill. (**C**) Pill gradually dissolves in water as indicated by the intensity of the blue dye for 4 min. Scale bar, 5 mm"* (Ze et al., 2022). Figure and caption obtained from Ze et al. (2022).



(c) *"[...] FEA results showing displacement and strain energy during shape morphing"* (Yang et al., 2021). Figure (cropped) and caption obtained from Yang et al. (2021).

robots degrade faster than conventional robots and, although their compliance makes them in some way more resistive to damages, the materials are less sturdy and more prone to tearing or other *traditional* modes of failure (see fig. 1.1c as an example). Therefore, lifelong adaptive techniques are essential for their sustained operation and usefulness. Alternatively, lifelong learning could benefit conventional robots, but assessing its effectiveness might take months or years. Soft robots offer a quicker and more economical way to test various methods.

1.2. Challenges

Successfully enabling lifelong learning faces a number of challenges. Some key challenges are addressed by Lesort et al. (2020), such as *catastrophic forgetting*, *handling memories*, and *detecting distributional shifts*.

Catastrophic forgetting relates to robots forgetting previously learned knowledge as they progress through life. Preferably, a robot should be able to quickly learn to adapt to new settings, without thereby 'overwriting' or otherwise erasing old knowledge.

Various ways of retaining old knowledge will be discussed, such as storing it somewhere for reusing it later. Storing memories can be done in many ways, and should aim to be as memory-efficient as possible, allowing for fast searches and retrieval of knowledge; another key challenge of handling memories.

As the goal is for a robot to adapt to changes throughout its lifetime, it is essential that it can detect these changes. This is related to detecting significant shifts in the data landscape it encounters. These shifts can be caused by imbalanced input data, a change in environment, the introduction of new tasks, or - the main focus of this study - the evolution of the robot's dynamics (Lesort et al., 2020).

1.3. Overview of the Study

Considering the aspects discussed, and the aim of the thesis, a research question can be formulated for this literature study:

> Which methods can be employed to accommodate or mitigate~~manage~~ the evolving dynamics due to wear and tear in the control of an agent over its operational lifespan?

To break down some key concepts this research question: *"methods"* is purposefully broad to leave the door open for potential traditional and non-machine-learning-based, although the main focus *will* be on machine-learning-based methods. *"the evolving dynamics due to wear and tear"* shifts the focus away from changing environments and tasks, but literature considering these applications will still be studied, as the techniques might be highly related. *"the control of an agent over its operational lifespan"* removes the focus from (soft) robotics, allowing the inclusion of techniques used in other fields that are still highly applicable, while emphasizing the lifelong nature. A brief overview of what is to come:

In **Chapter 2, 'Background and Definition'**, key concepts required for a clear understanding of the literature are presented and briefly explained.

In **Chapter 3, 'Blackbox Methods'**, general machine-learning methods that enable lifelong learning are explored.

In **Chapter 4, 'Learning Architectures'**, an overview of how various methods can be combined to create architectures capable of lifelong learning is presented.

In **Chapter 5, 'Storage and Retrieval'**, the focus is on the challenge of handling memories: how they are stored, retrieved and maintained.

In **Chapter 6, 'Discussion and Conclusion'**, the findings of the study are presented, resulting in an updated research question for the main thesis.

2

Background and Definition

Before exploring the literature on lifelong learning techniques, it is important to set appropriate definitions of several widely used terms, provide brief explanations of key concepts, and to give sufficient context for the scope of the research.

2.1. Lifelong, Continual, or Incremental Learning?

The broad concept of lifelong learning has been given many names throughout the literature. Common terms are (among others) continual, incremental, and online learning. There are some nuances between these terminologies when it comes to the exact details of how regularly models are updated (retrained), the batch sizes of the update training sets, and whether the data shifts over time. 'Lifelong' simply implies maintaining good functionality throughout the lifetime, 'continual' includes shifting data distributions, 'incremental' suggests at least regular updates with potentially single training samples, and 'online' refers to updates during test time. However, in this study, the terms above will be used interchangeably. Moreover, 'lifelong' will be used as a broad overarching term encompassing all the aspects above. Notably, using these terms, no distinction is made on whether they convey real-time implementations.

2.2. Learning vs. Adapting

Throughout chapter 1, it was mentioned several times that the goal is to have a robot continually adapt to changes. Apparently, 'adapting' and 'learning' can be used interchangeably in this context, or could (and will be throughout this study) both be interpreted as 'learning to adapt'. Adaptive learning is intricately tied to adaptive control, as the rise of data-driven models and machine learning has significantly changed the landscape of traditional adaptive control systems (Annaswamy & Fradkov, 2021). However, some distinction between the two can be made to provide some context.

Adapting to a change or disturbance does not have to directly involve learning methods. In fact, many techniques exist that can mitigate unforeseen disturbances, but these are automatic or 'hard-coded' processes and have not necessarily *learned* how to deal with the disturbances.

Any methods that involve tuning the parameters of models or control systems will be considered under 'learning'. This spans some implementations of traditional adaptive control (refer to section 4.1) to state-of-the-art reinforcement learning algorithms.

To provide an illustrative example: A classic PID controller can, if tuned correctly, nearly instantaniously adapt to disturbances, bringing a system back to the desired state. Consider now a mass-spring-damper system for which this controller has been tuned for position control, if the system parameters evolve, the PID controller will still eventually bring the system to the desired state, but in a less optimal manner (e.g. larger overshoots, over-/underdamped behaviour, etc.). In the context of this study, this would be considered adapting. If, however, the parameters of the PID controller were continuously updated to maintain ideal performance, that would be considered learning.

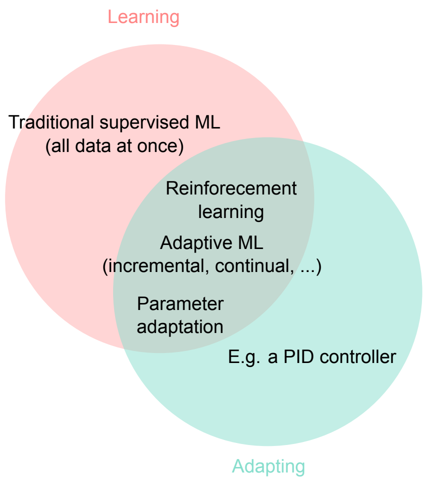


Figure 2.1: Venn diagram showing the differences and overlap between the concepts of ‘learning’ and ‘adapting’. In the intersection of the two, reinforcement learning relates to interaction-based methods (see section 2.3), adaptive ML encompasses (supervised) ML methods where models are trained on sequential batches of data (during deployment), and parameter adaptation ties ML into adaptive control (see section 4.1).

2.3. Learning Paradigms and Terminology

This section delves into various learning paradigms that are often considered when facing the lifelong learning challenge and explains some key terminology essential for understanding these paradigms.

Traditional supervised learning One might consider gathering an immense amount of data for several settings and let a traditional machine learning model loose on it. Unfortunately, this would require several key assumptions, e.g., the data will always be accessible for later updates; the imbalance of different settings in the data perfectly resembles that of the real world; when and how the data will shift is known exactly beforehand; and several other assumptions (Lesort et al., 2020).

Policy learning In this review, policies will simply relate to any decision-making process. For example, the~~The~~ policy can be a program, explicitly coded by a software engineer, or it can be a decision tree, or a machine learning model of any kind. In short, any process that in someway reviews the current situation and subsequently produces an action or strategy, will be considered a policy (Han et al., 2023; Ibarz et al., 2021; Nguyen-Tuong & Peters, 2011b).

Reinforcement learning Reinforcement learning is a goal-based learning method where interaction with the environment is key. It is akin to teaching a pet a trick, where good behaviors are rewarded with a treat, thereby encouraging the decision-making process to repeat the behavior or produce others like it.

In reinforcement learning, an agent interacts with an environment, observes the state it is in, and decides on an action to take. The specific state and action are valued using a reward function. Ultimately, the agent aims to obtain a good policy for interacting with the environment, e.g., finding an optimal sequence of actions to complete a task (Bhagat et al., 2019; Han et al., 2023; Zhang & Mo, 2021).

Distinctions can be made in the manner in which the policy is obtained. Policy-based methods learn a direct mapping from state to action, while model-based methods use a model of the agent and/or environment to predict the outcome of an action before executing it (Zhang & Mo, 2021).

Replay buffers In reinforcement learning, the agent learns through interaction with the environment. This setting requires at least temporary storing data obtained through interaction. This storage is usually called a replay buffer (Ibarz et al., 2021; Lesort et al., 2020).

3
Blackbox Methods

The term ‘blackbox’ in the context of lifelong learning techniques describes models that are inherently ‘data-driven’, requiring vast datasets to predict outcomes while often lacking in interpretability. These methods differ from traditional approaches, where clever architectural approaches or intuitive mathematical tricks might be exploited. Instead, so-called blackbox methods trade transparency for performance, as they process and learn from information in hard-to-interpret yet effective ways.

In this chapter, an overview is provided of how blackbox methods can be used as either building blocks for lifelong learning or to solve the lifelong learning challenge as complete solutions.

3.1. Learning Control Policies

A perfect example of a blackbox method is given by Alessi et al. (2023). In their study, they used Proximal Policy Optimization (PPO) (Schulman et al., 2017) to obtain a feedforward neural network (FNN) that can generalize over new dynamics and tasks. The controller is a blackbox: a desired trajectory, several measurements of the robot’s state, and other values of interest, such as the error of robot’s end-effector tip, are used as inputs, resulting in a control action. It essentially acts in a manner akin to a feedback controller using an inverse dynamics model, only in this case it is unknown what is happening *under the hood*.

The previous example is *“perfect”* in the sense that it perfectly embodies the simplistic nature of blackbox methods, however, it is worth noting that other techniques, such as more complicated architectures or combinations of multiple models, can be used in combination with these blackbox methods. Such architectural modifications will be discussed in chapter 4.

Apart from Alessi et al. (2023), several other examples of blackbox policies can be found in the literature.

In Zhou et al. (2022), two policies are learned for the control of a small humanoid robot. Exact details on the inputs and outputs of the controllers are not provided, but both policies are probabilistic in nature.

Thuruthel et al. (2019) use guided policy search (Levine & Koltun, 2013) over trajectory samples containing control actions for given regions in the state-space of a soft robotic manipulator. The resulting control policy is represented as a FNN which takes in current and previous states, as well as a desired target state, and returns the appropriate control action.

For the control of a soft robotic arm, in Nazeer et al. (2023), the policy is represented by a multi-layer perceptron (MLP) which takes in the arm’s current state[1] and outputs a control

[1]Rather, the sum of the state and a correction term, resulting from a separate network (section 4.3), but that is not

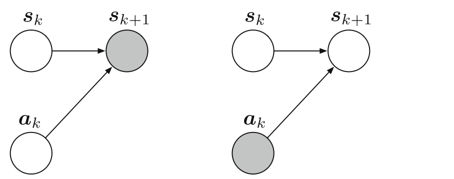
action. The policy is trained using the Soft Actor-Critic (SAC) (Haarnoja et al., 2018) reïnforcement learning algorithm.

In Lu et al. (2020), a more complicated technique is used. A probabilistic 'skill' distribution $p_\psi(z \mid s)$ and a neural network-based policy $\pi_\theta(a \mid s, z)$ are independently and simultaneously learned using SAC (refer to section 4.3). Both models are trained on rollouts of state transitions generated by a dynamics model. Once learned, a model-based planning algorithm called Model Path Predictive Integral (MPPI) (Williams et al., 2015) is used to find the optimal skill sequence in the skill distribution p_ψ, resulting in executable actions through the policy π_θ.

3.2. Learning the Dynamic Model

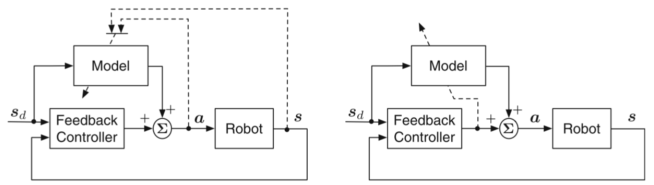
One way of having continually adapting control for soft robots is by implementing a (traditional) control scheme that uses a dynamic model (either forward or inverse) of the robot and then to incrementally update that model. The idea of learning a dynamic model for soft robots has advantages even outside the context of lifelong adaptation: many soft robots have such a large number of degrees of freedom and complex and nonlinear dynamics, that analytically deriving a dynamic model is both cumbersome and often insufficiently accurate (Gillespie et al., 2018; Giorelli et al., 2015; Kim et al., 2021; Thuruthel et al., 2019).

Blackbox methods also allow for learning both forward and inverse dynamic models (see fig. 3.1a) and can thus be used in several control schemes. When learning a forward model, the robot's actions and resulting states can be used as the inputs and outputs, respectively, this is so-called *direct modeling* (Nguyen-Tuong & Peters, 2011b, see fig. 3.1b). In the case of learning inverse dynamic models, such direct approaches might not always be applicable, due to the multiple solutions that exist for the inverse dynamics of robots with redundant degrees of freedom. Instead, an *indirect-modeling* approach can be taken, where the model is trained to minimize the error of the feedback controller (Nguyen-Tuong & Peters, 2011b).



(a) Forward model **(b)** Inverse model

(a) *"Graphical illustrations for different types of models. The* white nodes *denote the observed quantities, while the* gray nodes *represent the quantities to be* inferred. ***a** The forward model allows inferring the next state given current state and action.* ***b** The inverse model determines the action required to move the system from the current state to the next state* (Nguyen-Tuong & Peters, 2011b). Figure (cropped) and caption obtained from Nguyen-Tuong and Peters (2011b).



(a) Direct Modeling **(b)** Indirect Modeling

(b) *"**a** In the direct modeling approach, the model is learned directly from the observations.* ***b** Indirect modeling approximates the model using the output of the feedback controller as error signal [...]* (Nguyen-Tuong & Peters, 2011b). Figure (cropped) and caption obtained from Nguyen-Tuong and Peters (2011b).

The form that the dynamic model takes can vary. For example, in Gillespie et al. (2018), a neural network is trained to predict the next state of the robot, given the current state and a control action, the analytical gradients of these outputs w.r.t. the inputs are made available during the process of backpropagation and can be used ~~in matrices A and B~~ to form a more interpretable state-space system[2]. In Thuruthel et al. (2019) on the other hand, a forward model is learned for open-loop control, using the control action, the current state, and the previous state as input, and the next state as output to train a recurrent neural network. This forward model is kept in its more abstract blackbox form as a recurrent neural network (RNN),

relevant for the basic idea of the policy.

[2]The gradients are used in matrices A and B to obtain a linearized state-space system of the form $\boldsymbol{x}_{k+1} = A\boldsymbol{x}_k + B\boldsymbol{u}_k$

and is used in combination with trajectory optimization to generate trajectories in open-loop control.

Since the lifelong adapting nature of the techniques is embedded in the dynamic model, the choice of control scheme in which the learned dynamic model is used can vary from implementation to implementation. To illustrate, the aforementioned state-space system~~matrices A and B~~ from Gillespie et al. (2018) is used in a model predictive control setting, while the generated trajectories from Thuruthel et al. (2019) are used as samples to train a blackbox control policy (the dynamic model is in essence an intermediary step to create the blackbox policy mentioned in section 3.1, see fig. 3.2). More examples of several blackbox dynamic models and their control scheme can be found in the literature.

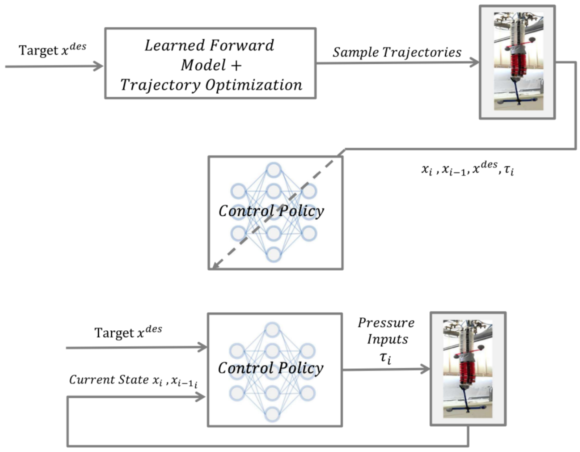


Figure 3.2: *"Block diagram describing the complete procedure for obtaining the closed-loop control policy (top). The learned control policy is encoded by a feedforward neural network and provided the appropriate closed-loop actions (bottom)"* (Thuruthel et al., 2019). Figure and caption obtained from Thuruthel et al. (2019).

Nazeer et al. (2023) represent a forward dynamic model of the soft robotic arm as an RNN model that uses the two preceding states and three preceding control actions to predict the next state. The model is then used with the previously discussed MLP policy (section 3.1).

Lu et al. (2020) was discussed in section 3.1 where it was briefly mentioned that training rollouts for the policies were generated using a dynamic model. This model is in fact also a data-driven model, being an ensemble of neural networks and predicting the next state s' based on the current state s and a control action yielded by the policy π_θ.

In and Nguyen-Tuong and Peters (2011a), the inverse dynamic model is represented by a Gaussian process (a type of probabilistic model) and is learned through Local Gaussian Process Regression (Nguyen-Tuong et al., 2008, 2009). The models are employed in a feedforward control scheme, supplemented by a feedback term that adjusts the control action outputted by the dynamic model.

To accurately capture the nonlinearities of an inverse dynamic model, Gijsberts and Metta (2011) want to use a kernel function to map inputs to a nonlinear feature space. However, due to the high computational cost of such a transformation, they approximate the kernel as a 'random feature' $z_\omega(x)$. This approximation $z_\omega(x)$ is a predefined nonlinear activation of a standard feedforward layer, but where the parameters $\boldsymbol{\omega}$ and β are sampled from specific distributions to obtain a good approximation of the desired kernel function. This makes tha architecture of the dynamic model essentially akin to a FNN, but with specifically initialized and frozen parameters for the very first layer.

Romeres et al. (2016) represent an inverse dynamic model of a robotic arm as a Gaussian process, but also experiment with using the arm's rigid body dynamics in several ways to enhance the model's performance. By incorporating the information from the robot's physical parameters into both the mean function and the covariance kernel of the Gaussian process, they develop semi-parametric models that are used in a feedforward manner where a feedback controller adds small corrections to the control actions outputted by the dynamic model.

Contrary to blackbox policy learning methods, the adaptation to new tasks is not included in the learning process of the dynamic model, but is entirely dependent on the control policy. In case control policies have been defined in advance for several tasks, these task settings might serve as a test to verify that the dynamic model generalizes well over these different settings. In situations where tasks are not known in advance, a control scheme such as an incrementally updated (inverse) dynamics model in some form of a feedback control loop might serve for simple applications (Nguyen-Tuong & Peters, 2011a). For more complex tasks, a more intricate control system might be warranted.

3.3. Incremental Learning

In section 3.1 and 3.2 an overview was provided of how blackbox machine learning techniques can be used to model either control policies or dynamic models (to be used in control policies). The choice of machine learning method can be important when it comes to the desired incremental nature of the learning techniques. To illustrate, consider these two main strategies in continually adaptive control:

1. All training is performed offline (after gathering sufficient data), and the robot is now completely capable of immediately adapting to changing dynamics it will encounter during its operational lifespan.

2. Training is performed incrementally as data comes in, which can potentially be done in real-time.

In both scenarios, the control of the robot would be continually adaptive. However, only strategy 2 aligns with the incremental objectives of the adaptive control challenge. In addition, the complex dynamics and many degrees of freedom of soft robots might make it difficult to accurately represent all possible degradations in dynamics (such as damages) or state-space configurations of the robot, potentially causing an imbalance in the training dataset of strategy 1. Learning incrementally to adapt the control policy or dynamic model is thus far more desirable, since it can cope with unforeseen circumstances.

Not all the previously discussed methods are - or have the possibility to be - implemented in an incremental manner. As an example, the two policies from Zhou et al. (2022) mentioned in section 3.1 are trained sequentially: one online and the other offline. Each of these policies required different training/optimization algorithms for their specific setting, in this case Maximum a Posteriori Policy Optimization (MPO) (Abdolmaleki et al., 2018) for the online interaction setting, and Critic Regularized Regression (CRR) (Wang, Novikov, et al., 2020) for the offline distillation setting (see section 4.2).

To learn the dynamic model from Gijsberts and Metta (2011), the authors implement Ridge Regression (a linear regression method that aims at keeping the parameters as small as possible) incrementally, whereas normally it is used in a batch-learning setting. They achieve this by employing a modified numerical approach based on the QR algorithm (Sayed, 2008), a technique in linear algebra that can process new information incrementally using Givens rotations (a process that is particularly efficient for online learning scenarios) (Björck, 1996). This enables real-time learning as new data comes in without recalculating the full model.

Lu et al. (2020) implement both offline and online training, with the potential for the entire method to be implemented online. In fact, it is well known that SAC - the algorithm used for updating the skill-space distribution and the policy - is very suitable for online updates. When it comes to the dynamic model, they mention that it is incrementally updated through interactions with the environment, but do not provide any details on the algorithm used for training the dynamic model.

In Nguyen-Tuong and Peters (2011a), the authors not only update their model incrementally, but they also manage to do it in real-time. This is achieved by using incremental Gaussian process regression algorithm, which uses only a select number of data points to fit a model. In addition, they introduce an efficient way to incrementally update the dictionary of datapoints in a way that it maintains the most informative points for a given budget. These two techniques are key to reducing the computational demand and allowing for real-time updates.

The semi-parametric models from Romeres et al. (2016) are learned using the Recursive Least-Squares algorithm (Landau et al., 2011), which inherently updates the model incrementally. Much alike Gijsberts and Metta (2011), this method uses a numerical trick from the field of adaptive filtering (Cholesky-based updates) to improve efficiency and enable updates in real-time.

3.4. Summary and Results

In table 3.1, a clear comparison of blackbox methods in lifelong learning is provided. This summary aims to give a straightforward reference, highlighting the strengths of blackbox methods and their role in lifelong learning.

Reference	Dynamic model	Control policy	Incremental
Gillespie et al. (2018)	SS-matrices	MPC	–
Gijsberts and Metta (2011)	FNN (with kernel approximation)	–	✓
Thuruthel et al. (2019)[3]	RNN	Open-loop + trajectory optimization	–
Lu et al. (2020)	NN ensemble	MPPI + NN	✓
Nguyen-Tuong and Peters (2011a)	Gaussian process	Feedforward	✓
Romeres et al. (2016)	Gaussian process	feedforward + feedback	✓
Nazeer et al. (2023)	RNN	MLP	[4]
Thuruthel et al. (2019)[5]	–	FNN	–
Alessi et al. (2023)	–	FNN	–
Zhou et al. (2022)	–	Probabilistic model	✓

Table 3.1: An overview blackbox methods and their ability to be implemented incrementally. For dynamic model learning methods, the control policy specifies the control scheme in which the model is used.

Some significant/promising results related to the representations of dynamic models and control policies in table 3.1 can be pointed out[6]. For example, Gillespie et al. (2018) provide a qualitative result of the dynamic model's performance (*Fig. 4* in Gillespie et al. (2018)) showing only minor deviations from the measured labels. The resulting MPC controller using said model is more robust to changes in the robot dynamics, as the model can be easily updated using new data.

[3]Used for generating training samples
[4]The incremental nature of the method is achieved by a third network, not by the dynamics model or control policy
[5]After training on generated samples
[6]Note that for some methods, the specific implementation of a dynamic model or control policy is more of an afterthought, as opposed to the framework in which they are used. Such results will be dicussed in later chapters.

The approach taken by Gijsberts and Metta (2011) was evaluated on several datasets relating to different robotic manipulators with varying degrees of freedom (1-7). The results show that the performance steadily improves with an increasing number of random features, outperforming other methods such as Gaussian processes from 1000 features onwards. Moreover, the method is the fastest to both train and predict of all methods compared in Gijsberts and Metta (2011), with a constant prediction time for training samples ranging up to 14000 samples.

The learned dynamic model from Nguyen-Tuong and Peters (2011a) is evaluated on a 7-DOF robotic arm, and is shown to be competitive with models obtained by other regression methods, which is significant due to its use of a much sparser dataset (section 5.4). The model is able to adapt to alterations in the dynamics of the arm (such as added mass to the end-effector) within a matter of *tens of seconds* (online).

Romeres et al. (2016) show that informing the blackbox Gaussian process regression with information of a 6-DOF robotic arm's physical parameters can slightly improve the performance of the model. Noteably, however, no qualitative results are provided for the claim of real-time applicability.

The performance of the dynamic model and control policy from Nazeer et al. (2023) are not directly compared to other implementations, but the improvements made by the use of their pecific architecture will be discussed in section 4.4.

No specific results are given for the accuracy of the dynamic model from Thuruthel et al. (2019), but the control policy resulting from the generated sample trajectories is shown to be robust to both dynamic changes and variations in control frequency, and sensory noise.

The controller fron Alessi et al. (2023) is able to generalize over environment dynamics as well as the dynaics of the robot itself. Futhermore, the polcy generalizes over new tasks and new observations.

The resulting distilled policy from Zhou et al. (2022) is shown to be more stable in both encountered environments, while maintaining better performance than pure MPO (used in online interaction). The advantages related to the specific implementation of their framework are discussed in section 4.4.

4

Learning Architectures

Blackbox methods for lifelong learning are promising approaches to achieve continually adaptive control, but are hard-to-interpret. What other techniques can be implemented that are more intuitive in nature?

Well-designed control and learning architectures can either enable or facilitate lifelong learning, whether the control schemes use blackbox models or more traditional methods. For example, some techniques use two or more blackbox models, where the idea is to have each model deal with a separate challenge of the lifelong learning goal. Other architectural methods aim to expand existing control schemes or policies to enable them to continually adapt.

In this chapter, these two main strategies will be further explored in the literature.

4.1. Traditional Adaptive Control

Adaptive control is about real-time control of uncertain dynamic systems through adaptation and learning, an idea that has been in continuous development since the 1950s (Annaswamy & Fradkov, 2021, section 2). The basic principle of adaptive control, is to adapt to environmental changes for better performance. This concept was first applied in engineering in the late 1950s, marking the inception of adaptive control systems, which monitor their own performance and adjust parameters accordingly.

Unlike conventional feedback control, which aims to mitigate disturbances acting on the *controlled* variables by referencing the desired values and simply changing the controller's *input* accordingly, an adaptive controller can deal with disturbances to the *controller's* variables and change the *parameters* of the controller. To further distinguish adaptive control from conventional feedback control, adaptive control is generally defined by three concepts, namely: measuring plant performance, a performance-based decision, and an adaptation mechanism (Landau et al., 2011, section 1.2).

The architecture and the interactions of these three main mechanisms can vary between implementations. For example, some systems use a reference model of the plant for performance measurements, such as in Model Reference Adaptive Control. Likewise, for adaptation mechanisms options include methods such as having a separate controller as an extension to the scheme, or several preset controllers to switch between (Landau et al., 2011, section 1.3).

Recent developments demonstrate the increasing use of machine learning techniques with traditional adaptive control concepts. Machine learning, particularly reinforcement learning, has enriched the adaptation mechanisms, enabling more intricate and nuanced performance-based decisions in adaptive control systems, reflecting a broader trend towards more learning-based control methods (Annaswamy & Fradkov, 2021, section 2).

4.2. Dual Architectures

Under 'dual architectures' the methods are gathered that have separate modules for solving different aspects of the lifelong learning challenge. The term 'dual' is used here in a very broad sense because each module might be a grouping of various techniques with intricate architectures. However, studying the literature, it can be found that most of the methods discussed here can be broken down into two main modules. Generally, these dual architectures often exist either of a policy/control module in combination with some type of knowledge base used for storing previous experiences and informing the policy (refer to chapter 5 for details on how knowledge bases can differ in implementation), or two (or more) control policies that are each in turn capable of handling low uncertainty and high uncertainty situations.

To illustrate how these modules can be a very broad concept, consider a knowledge base: A knowledge base must first and foremost store knowledge, but additionally also maintain and update this library. Furthermore, the knowledge base should be capable of categorizing new observations in relation to its library. All these capabilities are seldom performed by a single module, but the techniques used vary wildly between different implementations. It is therefore not very interesting to differentiate between each different implementation in this chapter, but rather use the overarching term 'knowledge base' and discuss the more intricate details in chapter 5.

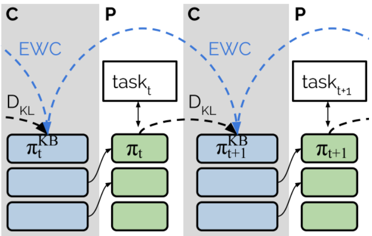
Having established the broad framework and significance of dual architectures, examples from literature that illustrate how these modular approaches have been effectively implemented can be discussed.

In Wang, Chen, and Dong (2020), a knowledge base stores potentially infinite sets of parametrized environments along with their respective control strategies. This repository is used to guide the current control policy: When a new environment is encountered, the environment models in the knowledge base are evaluated on how well they fit the observations. The control policy is then initialized using the policy parameters belonging to the most similar environment found in this knowledge base, and the policy continues to learn online. In addition, environment models can be added to the library when no suitable match to the observations can be found.

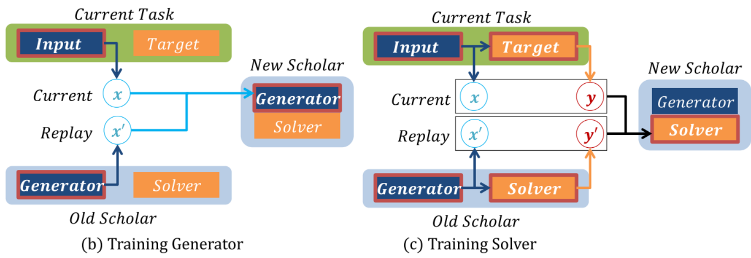
Schwarz et al. (2018) utilize an architecture consisting of two modules, which they name the knowledge base and the active column. During two alternating phases, progression and compression, the active column learns a new task and the learned policy is distilled into the knowledge base, respectively. The knowledge base in this case is a policy π_t^{KB} that serves as a stable reference for the active policy π_{t+1} during progression and remains unchanged at this time. During compression, the newly learned active policy is distilled into π_t^{KB} resulting in an updated policy π_{t+1}^{KB}, and the cycle can repeat (see fig. 4.1a).

To combat the issue of forgetting previous policies while training on new tasks, a common approach is to store old experiences and intermittently reuse them during the training process of a new policy. One downside is that this approach can be very memory-inefficient. Shin et al. (2017) therefore propose a method where a deep generative model, a so-called *Generator*, is trained to mimic the training data provided for a task-solving model called the *Solver*. The Solver is then trained on a mix of actual training data and generated samples provided by the Generator. Likewise, when a new Generator is trained to mimic the data of the next task, some samples created by the previous Generator are mixed in. This ensures that knowledge is not forgotten, since each Generator also learns to recreate data generated by its predecessor. A schematic of this process can be found in fig. 4.1b.

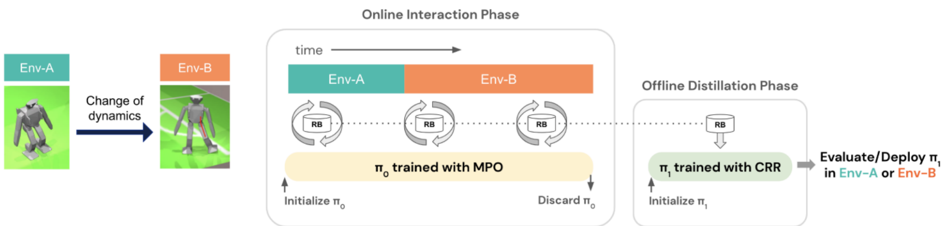
In Sprechmann et al. (2018), an embedding network f_γ and an output network g_θ learn to maximize the likelihood of a correct prediction from$\cancel{p(y \mid x,\gamma,\theta) =}$$g_\theta(f_\gamma(x))$, given the networks' parameters. The reason for the embedding-output architecture is the expansion of the embedding network with a memory M. This memory contains embeddings from f_γ and corresponding training labels, stored during training. At test time, the embedding network queries the memory for similar embeddings it might contain, and uses these to construct a context (refer to Chapter 5 for technical details). Based on this context, the parameters θ of the output network are adapted to obtain ~~the~~ more optimal parameters, resulting in better predictions ~~$\theta^x = \theta + \Delta_M(x, \theta)$, where $\Delta_M(x, \theta)$ is a learned adaptation based on the context provided by x~~.



(a) *"Illustration of the Progress & Compress learning process. In the compress phases (C), the policy learnt most recently by the active column (green) is distilled to the knowledge base (blue) while protecting previous contents with EWC (Elastic Weight Consolidation). In the progress phases (P), new tasks are learnt by the active column while reusing features from the knowledge base via lateral, layerwise connections"* (Schwarz et al., 2018). Figure (cropped) and caption obtained from Schwarz et al. (2018), see Chapter 5 for more technical details.



(b) *"Sequential training of scholar models. [...] Training a sequence of scholar models is equivalent to continuous training of a single scholar while referring to its most recent copy. (b) A new generator is trained to mimic a mixed data distribution of real samples x and replayed inputs x' from previous generator. (c) A new solver learns from real input-target pairs (x, y) and replayed input-target pairs (x', y'), where replayed response y' is obtained by feeding generated inputs into previous solver"* (Shin et al., 2017). Figure (cropped) and caption obtained from Shin et al. (2017), see Chapter 5 for more technical details.



(c) The 'offline distillation pipeline'. Figure obtained from Zhou et al. (2022).

An example of another approach taken when using dual architectures is found in Lu et al. (2019). Two modules, a model-free and a model-based module, work in synthesis to simulate the way human beings behave: functioning on *autopilot* most of the time and extensively planning only when we are uncertain. In the model-free module, a policy optimization algorithm (in this case, TD3 (Fujimoto et al., 2018)) uses the policy to propose an initial plan, while the standard deviation across an ensemble of value functions is used to provide a measure of uncertainty. Based on this uncertainty, a planning horizon H is set (short horizons for high uncertainty, and vice versa) and the model-based planner (MPPI) begins updating the initial plan for the given horizon, using the ensemble of value functions to evaluate its plans. During this model-based planning, any rollouts of potential plans are added to a replay buffer $\mathcal{D}_\pi$ which is subsequently used to train the TD3 policy.

In Zhou et al. (2022) the development of a policy exists of two phases: an online interaction phase and an offline distillation phase. Each phase employs a different learning algorithm. During interaction, a policy π_0 is trained online using MPO, which is good at exploring and quickly adapting to new environments, but does not necessarily counter forgetting. This interaction and simultaneous training happens over multiple environments or tasks, and data such as the state transitions, actions, and rewards, is gathered and saved in a replay buffer $\mathcal{D}$. Once finished, the policy π_0 is discarded, and a new policy π_1 is trained offline on buffer $\mathcal{D}$ using CRR, which is much better for generalization. This second policy is then deployed, and the agent is able to function in all environments encountered during interaction. Refer to fig. 4.1c for a comprehensive overview of the method. Intuitively, the data in the buffer reflects the strong adaptability from the online algorithm and is therefore a good guide for the second policy. Additionally, whilst the initial policy π_0 might have forgotten important experiences by the end of its training, the second policy can generalize over the data and thus counter forgetting, while also resulting in a more refined or distilled version of the initial policy.

4.3. Architectural Expansions

This section groups methods where an already functional control scheme or policy is enhanced by adding one or more modules to the scheme, enabling it to learn and/or adapt continually without actually altering the preexisting policy.

An example of such a scheme is given by Nazeer et al. (2023) (fig. 4.2). Their use of a blackbox control policy and a data-driven dynamic model was discussed in section 3.1 and 3.2. In this approach, the current state of the system is used to determine the next control action through the policy. This action then feeds into the dynamic model, predicting the system's next state. However, these models are static and do not adapt during operation. To ensure that the robot can continually adapt, an online regressing network is added in a feedback loop: With the physical robot as reference, the error between the actual state and the predicted state is measured. A buffer stores this error along with the policy's control action that produced these states. The online regressing network periodically trains on this buffer to predict the error that will result from a control input. This predicted error is added to the robot's actual resulting state to provide the policy with a mismatched state that will effectively result in a control action that compensates the error.

A planner of choice is used to control a robot in Chatzilygeroudis et al. (2016). However, once a significant drop in performance is detected (e.g., due to unexpected damage), the method employs various extra modules to counter this performance-drop. Based on the *Intelligent Trial and Error* algorithm (IT&E) (Cully et al., 2015), the authors use MAP-elites (Mouret & Clune, 2015, refer to Chapter 5) offline to create a behavior-performance map of atomic behaviors the robot could execute. Contrary to standard IT&E, the *performance* of a behavior is not a qualitative measure, but rather a Gaussian Process that describes the relative outcome of the behavior (termed the *Generic Reward* by the authors) and is aimed to be generic enough so that it is task-independent. A separate *Reward Selection Layer* (RSL) chooses the actual qualitative measure[1] to use as an evaluation of the *Generic Reward*.

As an illustrative example, a *Generic Reward* could be the relative position after executing an atomic behavior. The planner provides a target position to reach, and the RSL chooses the Euclidean distance to this point as a performance measure. Now, the behavior-performance map can be searched for the atomic behavior that will maximize the reward for the outcome of executing this behavior (i.e., the maximum reward of the *Generic Reward* of the behavior). The chosen behavior is executed (in the real world, or using simulation), the behavior and its outcome are used to update the Gaussian Processes, after which this process is repeated until a good compensatory behavior is reached.

In Lu et al. (2020), an agent initially learns a policy and a data-driven dynamic model. As discussed in chapter 3, the authors then expand on this basic setting by the inclusion of a skill-practice distribution $p_\psi(z \mid s)$. Related to this distribution is the skill-space itself, represented by an abstract higher-dimensional space $z \in [-1, 1]^{\dim(z)}$, and a skill-discriminator $q_\nu(s' \mid s, z)$.

[1]The authors do not provide any details on *how* this is achieved

All trained independently, the skill-practice distribution $p_\psi(z \mid s)$ learns which skill to use given a certain state, a skill-discriminator q_ν learns to model the result of using a skill in a certain state, and policy $\pi_\theta(a \mid s, z)$ learns a control action. These models update online by sampling a state uniformly from a replay buffer and sampling a skill from p_ψ. The resulting control action by π_θ is used in the data-driven dynamic model to obtain the next state, and the transition (s, a, z, s') is added to the buffer. Discriminator q_ν updates based on the transition, and an *intrinsic reward*[2] is calculated, aimed to encourage distinguishable and learnable skills. p_ψ, π_θ, and q_ν are finally updated using soft actor-critic.
As discussed section 3.1, the purpose of this method is to allow an MPPI planner to plan in the skill-space, where it will result in a selection of skills the agent is “confident” in, and that will subsequently, through π_θ, result in more predictable and robust actions.

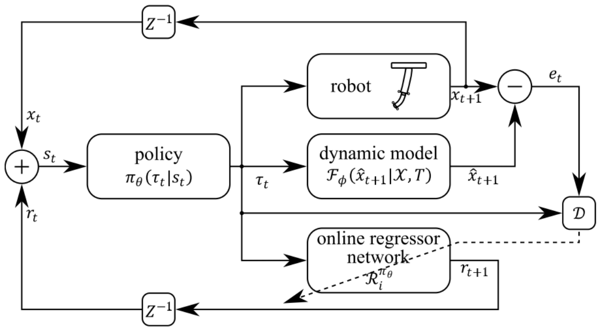


Figure 4.2: Policy adaptation control scheme. Figure obtained from Nazeer et al. (2023).

4.4. Summary and Results

Results on the performance of frameworks that use dual architectures of the form *knowledge base + policy* (i.e. Schwarz et al. (2018), Shin et al. (2017), Sprechmann et al. (2018), and Wang, Chen, and Dong (2020)) will be discussed in section 5.4.

The combination of a model-free and model-based modules from Lu et al. (2019) was evaluated on several lifelong learning environments (with changing task settings, and changing environments) and compared to common benchmarks (e.g. MPC, PPO). Te results show that their method succeeds in performing better or on par with the benchmarks, while only planning for a fraction of the timesteps compared to the benchmarks (making it more computationaly efficient).

Zhou et al. (2022) show that the policy resulting from an online interaction phase (using MPO) and subsequent distillation phase (using CRR) outperforms both a pure MPO and CRR policy in the environments encountered. Furthermore, the robustness of the distilled policy in both environments (during deployment) shows that the distillation process is a good solution to maintain both forward and backward transfer of experiences encountered in the interaction phase.

In Nazeer et al. (2023), the authors compare the performance of their control scheme without the online regressing network to the same scheme with the network for several trajectory tracking problems for a soft robotic arm. The scheme with the online regressing network significantly outperforms the scheme without the network in all cases. Additionally, the authors show that the expansion allows the policy to adapt to changes in dynamics, bridge the gap between training and deployment, and mitigate unforeseen external factors in the environment.

The use of a *Generic Reward* and *Reward Selection Layer* by Chatzilygeroudis et al. (2016) is compared to two variants of traditional IT&E algorithms. The authors evaluate the methods

[2]This is a skill-specific reward $r(s, z, s')$ that the policy is trying to oprimise, as opposed to a more traditional $r(s, a)$.

on a six-legged robot that has to navigate to a certain position, but has had one leg removed. The results show that the improved method is able succeed in the task using significantly fewer trials than the other methods, finding good compensatory behaviours in half the number of iterations than the other two.

Lu et al. (2020) show that their method of planning in the skill-space has significant advantages over other action-based planners. Comparing performance over several lifelong learning benchmarks, they show that skill-space planning allows for long-horizon planning, whereas traditional MPC is not stable in this setting. Furthermore, as agents update during deployment, the skill-space planning frame work is shown to be very stable during these changes, whereas e.g. pure SAC is not.

5

Storage and Retrieval

In section 4.2, many of the methods discussed employ a dual architecture where one of the modules is somehow responsible for retaining previous experiences, often termed the ‘knowledge base’. It was also mentioned that the general idea of using a knowledge base might be shared amongst many lifelong learning techniques, but the implementation varies wildly between them. A knowledge base does not always consist of a single module, but can have a complex architecture in and of itself; something akin to a library where the knowledge is stored, and a librarian who can organize and retrieve the knowledge correctly.

This chapter will aim to provide an overview of the many different shapes and forms such libraries can take, as well as the manner in which librarians might keep them organized.

An important note is that throughout this chapter, notations for vectors, buffers, policies, etc. will vary from paragraph to paragraph. This is because the notation is adopted from the notation used in the literature, aiming to reflect the ideas of the authors as they intended it.

5.1. Knowledge bases

In this section, a brief overview on the type of knowledge that is stored (e.g., a policy itself, state transitions), and the form in which they are stored (e.g., tuples, abstract representations).

In Nguyen-Tuong and Peters (2011a), the authors use Gaussian process regression to learn an inverse dynamic model (section 3.2). Since they want the model to incrementally learn in realtime, they limit themselves to a fixed budget of data points, represented as vectors $\boldsymbol{d}_i$, to use for updating the model. These data points are stored in a dictionary $\mathcal{D} = \{\boldsymbol{d}_i\}_{i=1}^{m}$ which is incrementally updated with the aim to be as informative as possible (section 5.2, 5.3). The contents of a point $\boldsymbol{d}_i$ are largely dependent on the application. Since Nguyen-Tuong and Peters (2011a) are trying to fit a model for a regression task, they opt for vectors of the form $\boldsymbol{d} = [\boldsymbol{x}\ \boldsymbol{y}]^T$, where $\boldsymbol{x}$ is simply the input vector and $\boldsymbol{y}$ are the target values.

In section 4.3 it was mentioned that Chatzilygeroudis et al. (2016) created a map of atomic behaviors an agent might execute. This map is created using Map-elites (Mouret & Clune, 2015), an algorithm that can set up a knowledge base containing many descriptions of solutions and recording their performance according to a specified performance measure. The knowledge base is represented by a discretized space of N user-defined dimensions. These dimensions define a feature-space of interest within a potentially huge or even infinite search space. The paper provides a conceptual example: *“MAP-Elites could search in the space of all possible robot designs (a very high dimensional space) to find the fastest robot (a performance criterion) for each combination of height and weight”* (Mouret & Clune, 2015)(fig. 5.2b). It can be noted that “search” can be interpreted as “store” in the context of the algorithm. The *description* of a solution can be stored in any way that suits the application, the performance metric is stored as a function. Chatzilygeroudis et al. (2016) provide a toy example for an agent that can move in 2D space using a preset step size; the *description* of the behavior here is thus a single value θ that gives the direction of the step. The evaluation metric in this case is the *Generic Reward* of the behavior (section 4.3), namely two Gaussian processes describing the relative position of the agent after executing the behavior.

To accomplish a good performance over several different classification tasks, Aljundi et al. (2017) train multiple models: one per task. These models, termed 'Experts', are stored in the knowledge base, in a manner most fitting for the model type (e.g., a FNN as layer architectures with corresponding weights and biases). The purpose of this knowledge base is to identify a fitting model for a given task and only selectively load this model into memory. Furthermore, when a new task is encountered, the most relevant 'Expert' can be loaded in to function as a prior for training a new 'Expert'.

As described in section 4.2, Sprechmann et al. (2018) introduce memory-based parameter adaptation, where a memory module M provides context to a network g_θ, allowing it to appropriately adapt its parameters for better performance. This memory contains key-value pairs (h_j, v_j), obtained during training and given by the embedding networks output and the training labels, respectively, i.e., $h_j = f_\gamma(x_j)$ and $v_j = y_j$. This means that the keys are represented by abstract feature vectors, the values are dependent on the task (e,g., one-hot encoded class labels for classification, target values for regression), and memory M is thus a higher-dimensional space spanned by these keys and values.

Schwarz et al. (2018) use a knowledge base in the form of a separate policy π^{KB} fig. 4.1a. This policy would thus be a neural network and is obtained through knowledge distillation (Hinton et al., 2015), a process where the outputs of one model are used as labels to train a second model. Active policies can reference the knowledge base to accelerate learning (section 5.2). When it is time to store a newly learned policy π_t, the outputs of the knowledge base's policy π_{t+1}^{KB} are trained to approximate the outputs of the active policy π, whilst also referencing π_t^{KB} to prevent drastic forgetting of old policies (section 5.3).

The knowledge base in Wang, Chen, and Dong (2020) stores a potentially infinite number of pairwise parameters $(\boldsymbol{\theta}, \boldsymbol{\vartheta})$, where $\boldsymbol{\theta}$ are the policy parameters, and $\boldsymbol{\vartheta}$ represents the parametrized environment. These pairs are assigned to an environment cluster $M^{(l)}$, the purpose being to recognize the most similar environment model compared to the one currently encountered, and to use the corresponding policy parameters as a strong initial policy to fine-tune (section 5.2). The form of the policy parameters $\boldsymbol{\theta}$ again depends on the form of the policy itself. The environments, however, can be parametrized as models by training the reward function $r = g_{\boldsymbol{\vartheta}}^1(s, a)$ and the state transition function $s' = g_{\boldsymbol{\vartheta}}^2(s, a)$ and concatenating them to obtain $[r, s'] = g_{\boldsymbol{\vartheta}}(s, a)$. An environment model $g_{\boldsymbol{\vartheta}}$ can be trained on sequences of state-action pairs to produce corresponding reward-state sequences, and the corresponding parameters $\boldsymbol{\vartheta}$ are stored alongside $\boldsymbol{\theta}$ in the library.

In Xie and Finn (2021), previous experiences are stored in the form of tuples $(\boldsymbol{s}, \boldsymbol{a}, \boldsymbol{s}', r)$. These tuples are stored in a replay buffer $\mathcal{D}^i$ for each task $\mathcal{T}^i$. For *all* tasks, each replay buffer is stored; they will serve to provide offline data for pretraining an agent when learning a new task (section 5.2).

Similar to Schwarz et al. (2018), the authors in Shin et al. (2017) do not store knowledge explicitly in a buffer or library, but rather use a separate model to fulfill this function. However, contrary to Schwarz et al. (2018), the model is not a policy but a generative model trained to imitate training data from previous tasks. The generative model used in this case uses a Generative Adversarial Network (GAN) architecture. These models have a dual architecture of their own, where a generating model is trained to optimize producing outputs that can *fool* a discriminator, which is in turn trained to optimize distinguishing real data from generated data. These two networks are trained in parallel and compete against each other (hence, adversarial) to obtain the best result.

5.2. Retrieval and Detecting Distributional Shifts

Storing knowledge is not very useful in and of itself. To properly exploit the advantages of using a knowledge base, knowledge needs to be retrieved. This encompasses recognizing what might be relevant knowledge given new observations, using the knowledge base to detect when an agent is 'breaking new ground' in the data landscape, or the manner in which all information in the knowledge base generally can be used to inform an agent.

Dictionary $\mathcal{D}$ in Nguyen-Tuong and Peters (2011a) is simply used as a selective representation of the training data to fit the model using Gaussian process regression, and, being dynamically updated (section 5.3), thus informs the model by providing it with the most informative data points. However, a technique used to measure the informativeness of a point touches on detecting a distributional shift. The authors use a linear independence test to check whether a potentially new datapoint $\boldsymbol{d}_i$ can be approximated in the space spanned by all the vectors $\{\boldsymbol{d}_j\}_1^{i-1}$. Figure 5.1a shows how the measure δ is the distance from point $\boldsymbol{d}_i$ to the (hyper-)plane $\{\boldsymbol{d}_1, \ldots, \boldsymbol{d}_{i-1}\}$. A predetermined threshold can then be used to decide whether point $\boldsymbol{d}_i$ relates to a distributional shift.

In the context of pure MAP-elites from Mouret and Clune (2015), a stored solution is retrieved by querying the specific features of interest. The authors call their method an *illumination* algorithm, rather than a search algorithm, since MAP-elites does not search for a single optimum, but instead 'illuminates' the best solutions in the defined feature space. In Chatzilygeroudis et al. (2016), the behavior performance-map (created using MAP-elites) is searched using Bayesian Optimisation (Brochu et al., 2010). Bayesian Optimisation uses the Gaussian processes that predict the *Generic Rewards* as its prior distribution and employs the Upper Confidence Bound (UCB) (Brochu et al., 2010) function as its acquisition function to guide the selection process, balancing exploration and exploitation. The key addition to this method is that the acquisition function does not directly evaluate the prior distribution's Gaussian processes, but rather the reward function, provided by the *reward selection layer*, of those Gaussian processes.

The manner in which the collection of experts in Aljundi et al. (2017) is used is twofold. During training, the most relevant expert is selected as a prior for the new model by measuring task relatedness. At test time, the collection of experts is simply used to select the best expert for the task at hand. To measure task relatedness without having to store all the training data of previous tasks, the authors train an auto-encoder[1] network A_k alongside each expert. This auto-encoder learns an abstract representation of task data, where important nuances in the task's characteristics can be more accentuated. The reconstruction error between the validation data D_k of a task T_k and a previous auto-encoder's A_a reproduction is used to measure how well T_k resembles T_a. The expert corresponding to the most related task is selected as a prior model. Depending on how close the reconstruction error of A_k is to that of the most relevant A_a, a transfer method of choice can be selected, such as simply initializing the parameters and fine-tuning for low-related tasks, or more complicated methods like learning-without-forgetting (Li & Hoiem, 2018) for highly related tasks.
At test time, the same mechanism is effectively employed to select the most relevant expert, with the addition of a softmax layer: a sample x is fed through all auto-encoders and the softmax layer, comparing their reconstruction errors, and selecting the most likely expert.

[1]An auto-encoder is an embedding-decoding network that is trained to produce outputs similar to its inputs, regularization criteria prevent it from simply copying its input (Aljundi et al., 2017)

The memory M in Sprechmann et al. (2018) stores key-value pairs $(f_\gamma(x_{train}), y_{train})$ obtained during training. At test time, a query q is made by calculating the embedding $f_\gamma(x)$. Using K nearest neighbors (KNN), a context C is constructed. This context consists of the retrieved keys $h_k^{(x)}$, values $v_k^{(x)}$, and KNN weights $w_k^{(x)}$, for $k = 1, \ldots, K$. An adaptation Δ_M is temporarily fine-tuned to adapt~~minimize the weighted average negative log-likelihood over the context, i.e.:~~ the parameters of output network g_θ with the goal~~are adjusted as~~ to reduce the error for the similar examples in the context, thereby improving its prediction of the current sample. An important note to this is that the adaptations Δ_M are accumulated in Δ_{total} which is permanently added to the parameters, thus continually improving the network's performance throughout test time until Δ_M eventually diminishes.

In Schwarz et al. (2018), the knowledge base policy π^{KB} informs an active learning policy during the so-called *progress* phase. This transfer of knowledge is achieved through layer-wise connections: for a given layer i in the active policy, activations h_i are computed using the previous layer's activations h_{i-1} and the activations of the corresponding knowledge base's layer h_{i-1}^{KB}. These knowledge base activations are first fed through an 'adapter' in the form of a multi-layer perceptron so that the extent to which activations from π^{KB} are added can be precisely fine-tuned during training, without altering π^{KB} itself.

When an agent encounters a new environment M_t in Wang, Chen, and Dong (2020), it first samples some transitions (s, a, r, s') using a uniform behavior policy, ensuring broad exploration of the environment. These transitions are used to construct state-action pairs and reward-state pairs used as inputs $\boldsymbol{X}_t$ and labels $\boldsymbol{Y}_t$, respectively (section 5.1). The predictive likelihood $p_{\boldsymbol{\vartheta}_t^{(l)}}(\boldsymbol{Y}_t|\boldsymbol{X}_t)$ is then calculated for each model $\boldsymbol{\vartheta}_t^{(l)}$, measuring how well a model explains the observations. This way, the identity l^* of the current environment is obtained. The policy parameters $\boldsymbol{\theta}_t$ are then initialized as $\boldsymbol{\theta}^{(l^*)}$, ready to be further fine-tuned.

In Xie and Finn (2021), knowledge transfer happens by simply pretraining a policy on old replay buffers. The buffers of previous tasks $\mathcal{D}^{1:i-1}$ are restored into a buffer $\mathcal{D}^{\mathrm{src}}$ when training on a new task $\mathcal{T}^i$. However, these buffers are relabeled using the new reward function from the current environment[2]. A sample $(\boldsymbol{s}, \boldsymbol{a}, \boldsymbol{s}', r) \in \mathcal{D}^j$ is thus relabeled as $(\boldsymbol{s}, \boldsymbol{a}, \boldsymbol{s}', r^i(\boldsymbol{s}, \boldsymbol{a}))$ (see fig. 5.1b).

The generative model and task solving model from Shin et al. (2017) are combined into a module termed the *Scholar*. Each task requires the training of a new *Scholar*, which is informed by the *Scholar* of the preceding task: Along with the input data x and labels y for the task at hand, the current task solving model also receives generated samples x' from the old *Generator*, labeled by outputs y' from the corresponding old *Solver* (see fig. 4.1b).

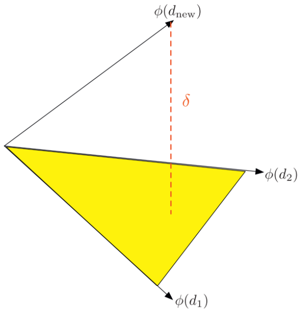
5.3. Maintaining the Knowledge Base

Maintaining the knowledge base can ensure that it remains informative. A knowledge base can be updated by adding new data points, by adapting existing data points, or - if applicable - updating a network or policy. Furthermore, these updates could be performed dynamically throughout the application's operational lifetime, or during separate phases only, e.g., updating the knowledge base during training and keeping it static at test time.

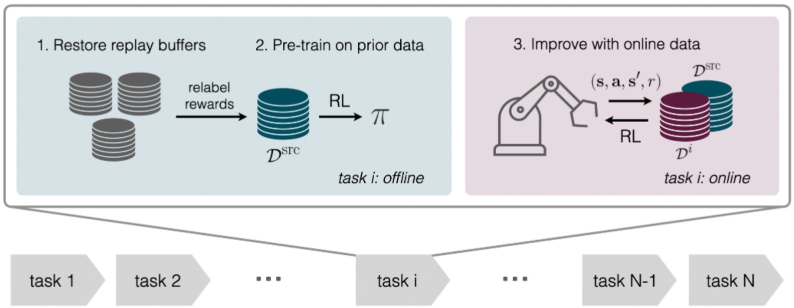
Of the previously discussed literature, both Aljundi et al. (2017) and Sprechmann et al. (2018) do not update their knowledge bases after training. Sprechmann et al. (2018) mention that during training, a fixed budget memory is used, where the oldest data is replaced first, once capacity is reached.

In Nguyen-Tuong and Peters (2011a), the linear independence test results in measure

[2]A key assumption of their method is that the reward functions of environments are either known beforehand, or are user-specified



(a) *"Geometrical interpretation of the independence measure δ. Here, the dictionary consists of two data points $\{\boldsymbol{d}_1, \boldsymbol{d}_2\}$ which span a linear plane in the feature space. The independence measure δ for a new data point $\boldsymbol{d}_{new}$ can be interpreted as the distance to this plane."* (Nguyen-Tuong & Peters, 2011a). Figure and caption obtained from Nguyen-Tuong and Peters (2011a).



(b) The process of relabelling rewards on old buffers and subsequent improvement through onlne interaction. Figure obtained from Xie and Finn (2021)

δ, which is used to update the dictionary with fixe budget. To reiterate, the δ for a point $\boldsymbol{d}_i$ measures its distance to the space spanned by all the points currently in the dictionary (fig. 5.1a and 5.2a). Conversely, a smaller δ can be better approximated by the dictionary. If dictionary $\mathcal{D}$ is not yet at full budget, and the δ for a new point $\boldsymbol{d}_i$ exceeds a certain predetermined threshold, the point should be added to the dictionary. However, when the dictionary is at full capacity, the new point is inserted in the dictionary, which is then re-evaluated to find and delete the now least informative point. Note that after any insertion or replacement of a dictionary point, *all* points in $\mathcal{D}$ have to be evaluated for linear independence again, since adding or removing points changes the independence of other points as well.

The map in Chatzilygeroudis et al. (2016) is constructed in simulation using MAP-elites. An empty N-dimensional grid map, spanned by the N features of interest, is initiated see fig. 5.2b. As a start, a certain number of behaviors are randomly generated and placed in the map according to their feature descriptors. In simulation, the performance measures (in this case, the *Generic Reward*) of these *solutions* are recorded and stored alongside the behaviors in the map. For all future iterations, the rest of the map is gradually filled in by selecting a solution in the map at random, and creating a modified version of it (e.g., by mutation or crossing-over multiple solutions). This modified version is placed in the map according to its features, and the performance is stored alongside it. Should a cell in the map already be occupied, then only the solution with the best performance is stored[3]. This process can repeat for a predetermined number of iterations, or until a solution with a certain satisfactory performance is obtained (Mouret & Clune, 2015).
During deployment, when searching the map for a good behavior (section 5.2), the selected behavior and its result in the real world are recorded and used to update the Gaussian processes that function as performance measures in the map (Chatzilygeroudis et al., 2016).

Updating policy π^{KB} from Schwarz et al. (2018) is done through knowledge distillation. At *compression* time t, policy π_t^{KB} is trained to produce outputs similar to active policy π_t by minimizing the KL loss (Kullback & Leibler, 1951) between their respective outputs. However,

[3]In Chatzilygeroudis et al. (2016), the performance is actually a set of Gaussian processes that model the relative outcome of a behavior, but the authors do not specify on what basis solutions in identical cells of the map are replaced

to make sure that previous knowledge is not forgotten, the loss function is expanded by a term that uses Elastic Weight Consolidation (EWC) (Hinton et al., 2015). This term essentially 'punishes' large differences between the parameters θ_t^{KB} and θ_{t-1}^{KB}.

In Wang, Chen, and Dong (2020), the library of environment parameters is updated incrementally. At each time t, the library might contain L sets of parameters $\{\boldsymbol{\theta}_t^{(l)}, \boldsymbol{\vartheta}_t^{(l)}\}_{l=1}^{L}$ corresponding to environment clusters $M^{(l)}$. First, a new set $(\boldsymbol{\theta}_t^{(L+1)}, \boldsymbol{\vartheta}_t^{(L+1)})$ is initialized to represent a potential new environment cluster. After gathering transitions and calculating (preliminary) predictive likelihoods of the models, posterior probabilities of model parameters w.r.t. transitions are calculated, i.e., $P(\boldsymbol{\vartheta}_t^{(l)}|\boldsymbol{X}_t, \boldsymbol{Y}_t)$. However, the posterior for $\boldsymbol{\vartheta}_t^{(L+1)}$ is actually calculated differently than for all $l \leq L$. The calculation is based on a Chinese restaurant process (CRP)[4], where the probability of the environment cluster being added is considered in its evaluation on the transitions. Likewise, the posteriors of the models $\boldsymbol{\vartheta}_t^{(l)}$ for $l \leq L$ are in turn calculated considering the number of models that have been assigned to their respective clusters. If the posterior of $\boldsymbol{\vartheta}_t^{(L+1)}$ is more likely than for any $\boldsymbol{\vartheta}_t^{(l)}$ (with $l \leq L$), the library is expanded with parameters $(\boldsymbol{\theta}_t^{(L+1)}, \boldsymbol{\vartheta}_t^{(L+1)})$.
Expansion or not, all environment model parameters in the library are then updated to $\boldsymbol{\vartheta}_{t+1}^{(l)}$ using the Expectation-Maximisation (EM) algorithm. This updated library is used as in section 5.2 to identify the current environment. After fine-tuning the selected $\boldsymbol{\theta}_t^{(l^*)}$, resulting in $\boldsymbol{\theta}_t^*$, all policy parameters are updated to $\boldsymbol{\theta}_{t+1}^{(l)}$, which effectively results in identical parameters except for $\boldsymbol{\theta}_{t+1}^{(l^*)}$. A graphical overview of the entire method can be found in fig. 5.2c.

In Xie and Finn (2021), for the online improvement of the pretrained policy, the agent stores its interactions with the environment in buffer $\mathcal{D}^i$. To update the policy parameters, the agent bases itself on a randomly sampled batch from $\mathcal{D}^{\mathrm{src}} \cup \mathcal{D}^i$. When the agent reaches a sufficient performance, $\mathcal{D}^{\mathrm{src}}$ can be discarded and $\mathcal{D}^i$ is saved somewhere (to be restored and relabeled for future tasks, section 5.2).

To transfer knowledge "stored" by old generative models in Shin et al. (2017), new *Generators* are not only trained to imitate real input data from the task at hand, but also to imitate "fake" data produced by old models. The exact ratio at which real new samples and old fake samples are mixed is an important aspect that influences the rate at which new *Scholars* will forget data from older models (see fig. 4.1b).

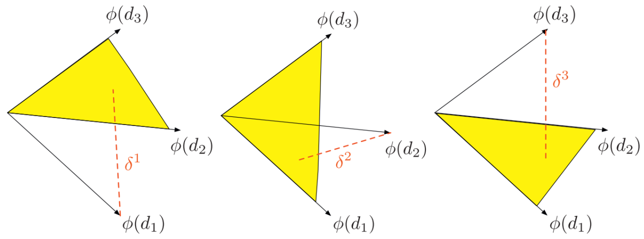
5.4. Summary and Results

In table 5.1, a comprehensive overview of the methods discussed in the literature is presented, which can serve as a quick reference for the methods discussed.

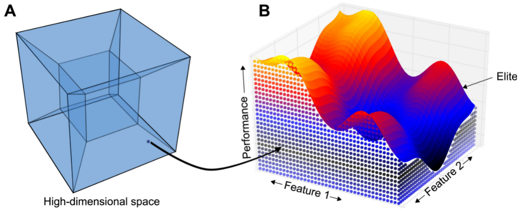
As discussed in section 3.4, Nguyen-Tuong and Peters (2011a) achieve competetive or better results in modeling the dynamics of a 7-DOF robotic manipulator compared to otherregression or local learning methods. They achieve this performance using a dictionary containing only 2000 datapoints, showing that there selection methods is effective at gathering only the most informative points. Additionally, they mention that the method is fast enough to be used in real-time applications.

The method used to create the knowledge base from Chatzilygeroudis et al. (2016) is the MAP-elites algorithm by Mouret and Clune (2015). In their original paper, the authors show that MAP-elites proves to have the best performance over a variety of criteria (such as reliability, precision, and coverage) as well as tasks (such as designing neural networks and soft-robot morphologies) compared to other search algorithms. For retrieving the atomic

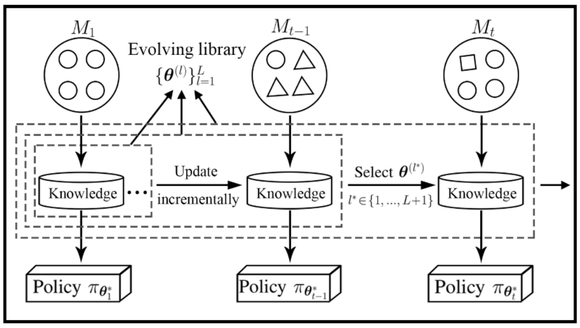
[4] In the Chinese Restaurant Process (CRP), customers choose tables in a restaurant, symbolizing cluster formation. They either join an existing table based on its occupancy or start a new table, influenced by a concentration parameter (Pitman, 2006; Wang, Chen, & Dong, 2020)



(a) *"Geometrical interpretation of individual independence measures $\delta^i = 1, 2, 3$ for the dictionary $\mathcal{D} = \{\boldsymbol{d}_1, \boldsymbol{d}_2, \boldsymbol{d}_3\}$. The dictionary point with a minimal δ^i is more likely to be deleted. After every dictionary operation (e.g., insertion and deletion of dictionary points), the individual independence measures δ^i have to be incrementally updated"* (Nguyen-Tuong & Peters, 2011a). Figure and caption obtained from Nguyen-Tuong and Peters (2011a).



(b) Graphical representation for a MAP-elites search space and grid map. Figure obtained from Mouret and Clune (2015)



(c) *"[...] Lifelong incremental RL. $M_t \in \mathcal{M}, t = 1, 2, ...$ denotes the specific MDP*[a]*/environment at time period t, and D denotes the dynamic environment over the MDP space $\mathcal{M}$, and $\boldsymbol{\theta}$ denote learning parameters"* (Wang, Chen, & Dong, 2020). Figure and caption obtained from Wang, Chen, and Dong (2020)

[a]Markov decision process

behaviors, Chatzilygeroudis et al. (2016) use Bayesian optimization, which is shown to be an improvement over the original IT&E algorithm (section 4.4).

Aljundi et al. (2017) compare their method against several common approaches (single jointly-trained model, multiple finetuened models, multiple *learning-without-forgetting models*, ...) on the sequential learning of classification tasks. They show that the auto-encoder based task relatedness measure is highly effective at selecting the most relevant expert for a given task, and that their method is superior to more common sequential-training approaches. Likewise, they are able to compete with other multiple-model approaches.

The implementation of memory-based parameter adaptations from Sprechmann et al. (2018) is shown to be effective in a number of tasks, from classification to language modeling. The memory also allows for continual adaptation to shifts in the data (new, unseen classes, etc.) as well as being robust to unbalanced data during training.

Schwarz et al. (2018) test their *progress & compress* framework on several tasks ranging from Atari games to maze navigation. They show that the distillation process with EWC maintains good negative transfer of knowledge (i.e. *not* forgetting old tasks). Improvements in positive transfer of knowledge (through layer-wise connections) are less significant, but still competetive with other methods if the weights of the new *active column* are reinitialized.

In Wang, Chen, and Dong (2020), the authors show that their method can indeed correctly (as in, in an optimally useful manner) cluster environments, as well as correctly identify the current environment as one previously seen. Moreover, they show that the policies initiated from these corresponding environments outperform (after fine-tuning) cutting edge methods in

Reference	Storage	Retrieval	Maintenance
Nguyen-Tuong and Peters (2011a)	input-output vectors	training on knowledge base	incrementally expanded
Mouret and Clune (2015)	solution features + performance	–	static
Chatzilygeroudis et al. (2016)	behavior features + GP of outcome	Bayesian optimisation	incrementally updated (GPs)
Aljundi et al. (2017)	Collection of neural networks	auto-encoder reconstruction error of input data	–
Sprechmann et al. (2018)	embeddings + labels	KNN to query	static
Schwarz et al. (2018)	policy	layer-wise adapters	distillation + EWC
Wang, Chen, and Dong (2020)	policy & environment parameters	predictive likelihood on transitions	incrementally expanded + updated
Xie and Finn (2021)	transition tuples	training on (relabeled) knowledge base	incrementally expanded
Shin et al. (2017)	generative model	training on generated data (*Solver*)	training on generated data (*Generator*)

Table 5.1: A summary of knowledge base implementations in the literature, specifying the form of the knowledge, how it is used to inform an agent, and how the knowledge base is maintained. Note: for maintenace, “expanding” relates to adding entries to a knowledge base, and “updating” relates to editing the entries.

continual learning over a range of tasks with dynamic environments (2D navigation, 3D agent locomotion, …).

In Xie and Finn (2021), the method is compared to common sequential learning approaches (such as fine-tuning) and more state-of-the art methods that reuse previous data. The tasks used to compare performance vary from 3D key insertion to rotating a valve in a certain direction. At all vale rotation tasks, they outperform the other methods, while for the key insertion tasks, they perform among the best state-of-the-art approaches, with only very slight differences in performance. Additionally, they find that online performance increases with the number of tasks previously trained on.

Finally, Shin et al. (2017) show that *generative replay* can indeed recover past knowledge by generating old data and outperforms simple sequentially trained models. They show that their method is applicable to the transfer of tasks, domains, and classes by enhancing a *learning-without-forgetting* model with their *generative replay* framework and finding in increased performance on all test settings. Lastly, they aknowledge that the choice and quality of the generator is a major factor to the performance.

6

Discussion and conclusion

The aim of this literature study can be summarized by the research question:

> Which methods can be employed to accommodate or mitigate~~manage~~ the evolving dynamics due to wear and tear in the control of an agent over its operational lifespan?

In chapter 3, blackbox solutions were presented that offered either incrementally adapted, or sufficiently flexible and robust control, but that are opaque in nature. Furthermore, the case was made for employing such solutions as a robust or adaptive dynamic model, which can then be combined in control schemes of choice. Some methods did not offer incremental updates, but rather resulted in highly flexible control solutions, however, these methods would then heavily rely on the key assumptions mentioned in section 2.3.

Chapter 4 focused on adaptive control schemes, common machine-learning-based architectures used in lifelong learning, and ways in which control methods were expanded to enable lifelong learning. Noteworthy is the common use of dual architectures, specifically the use of knowledge bases in combination with active policies, and the one-model-per-setting approaches.

In chapter 5, extra attention was given to knowledge base implementations. Implementations included, among others: separate policies, collections of models, libraries of model and environment parameters, abstract representations of input data, etc. A connection between retrieving knowledge, classifying observations, and detecting distributional shifts was established. The predominant use of dynamic libraries justified an overview of the mechanisms they employ for maintenance.

6.1. Promising Approaches

The use of data-driven dynamic models appears to lend itself well to various settings (see section 3.2), whether the model is used in combination with a blackbox policy or an adaptive control scheme (section 4.1). Using such a dynamic model can also constrain the problem to only incrementally updating the dynamic model, rather than the control policy, which aligns well with the focus on evolving dynamics instead of different tasks.

Generally, the implementations of dual architectures (section 4.2) provided more interpretable, or at least more intuitively understandable solutions, even though the use of blackbox methods is not excluded in these architectures. In particular, the use of knowledge bases section 5.1 provides an intuitive solution to the problem of catastrophic forgetting, and the study revealed that a knowledge base does not have to imply vast storage requirements. Additionally, dual architectures with knowledge bases, rather than separate models for different settings, reflect a strong preference towards a more 'learning' and 'remembering' related approach as opposed to 'rapidly adapting'. This preference can be beneficial in the setting of soft robots that have fragile parts which are often replaced.

Another attractive approach is that of an architectural expansion to an existing model (section 4.3), enabling lifelong learning. Like blackbox dynamic models, the methods constrain the lifelong learning problem to a select number of modules, but in this case there is more variety in where adaptions to the control are applied. Furthermore, these methods provide the option to leave the base control method unchanged.

6.2. Research Avenues

A number of topics that received less attention in te literature can be identified as potential avenues for future research.
For example, while many methods allow for incremental learning, whether through updates of a model or updates of a knowledge base, no emphasis is given to real-time applications (except for Nguyen-Tuong and Peters (2011a)). Much of the literature does not specify the computational speed of the methods they employ. Data is likely gathered in real-time during interactions with the environment, but the incremental updates can take an unspecified amount of time (in fact the video that Chatzilygeroudis et al. (2016) include with their research clearly shows the robot pausing after each trial).

The thesis will focus on the continually evolving dynamics of a soft origami robot. However, in the literature, a lot of focus is given to policies that can handle "discrete" changes in tasks and environment, where the dynamics or specifications change abruptly from one environment or task to the other. Even when learning methods are applied to the dynamics of (soft) robotic manipulators, the main reason often is to avoid analytically deriving the complex nonlinear models, and then to allow adapting to different environments and tasks. Section 1.3 mentioned that these methods would be reviewed because the techniques could be highly relevant, and so this could very well be a moot point, but the gap in the literature was noticeable.

Although the architectural expansion approach shows promise, each expansion is highly tailored to the base control scheme or policy it is attached to. This is generally the case for other approaches too, where the architectures, algorithms, and possible knowledge bases are all strongly linked to one another, leaving little room for flexibility in the approach. An interesting avenue might be to develop a *one-size-fits-all* expansion. Ideally such a solution could be applied to a wide range of control policies, schemes, or algorithms, without the need to alter them in any way.

6.3. Conclusion

Considering the foregoing sections, some key aspects are identified as desirable features for the implementation resulting from the main thesis:

- The solution should specifically deal with continually evolving robot dynamics. This would mean constraining the robot to executing a single (simple) task, in a single environment.
- The emphasis should be on "learning" and "remembering", rather than only adapting. This includes dealing with catastrophic forgetting and handling memories. A desirable result or application could be to replace a broken part of an origami robot, and have the control recognize the dynamics of the new replacement.
- The aim should be a *plug-and-play* module that can enable lifelong learning in a multitude

of different approaches to robot-control, given the most base assumptions that would hold for most control approaches (e.g., robot states as inputs, control actions as output).

- The architecture and functioning of this model should be intuitive and interpretable; however, this does not exclude the use of opaque blackbox methods within the module.
- Real-time adaptation and learning would be desirable.

Bringing these points together, an updated research question for the main thesis can be formulated.

> How can machine learning-based strategies be integrated into the control system of an origami robot to continuously adapt over its operational lifetime to the disturbances caused by evolving dynamics (wear and tear, part replacements, and damages) in its materials?~~How can a given control system for a soft origami robot be expanded to enable it to learn about its evolving dynamics, resulting from factors such as wear, damages and part replacements, and have it appropriately adapt over its operational lifetime?~~

~~This research question implies the desirability of a *plug-and-play* solution, and implies an emphasis on "remembering".~~ This question will provide a solid foundation, and will be used to guide the development of the main thesis.

List of changes

Bibliography

Abdolmaleki, A., Springenberg, J. T., Tassa, Y., Munos, R., Heess, N., & Riedmiller, M. (2018). Maximum a Posteriori Policy Optimisation. http://arxiv.org/abs/1806.06920

Alessi, C., Hauser, H., Lucantonio, A., & Falotico, E. (2023). Learning a Controller for Soft Robotic Arms and Testing its Generalization to New Observations, Dynamics, and Tasks. *2023 IEEE International Conference on Soft Robotics (RoboSoft)*, 1–7. https://doi.org/10.1109/RoboSoft55895.2023.10121988

Aljundi, R., Chakravarty, P., & Tuytelaars, T. (2017). Expert gate: Lifelong learning with a network of experts. *Proceedings - 30th IEEE Conference on Computer Vision and Pattern Recognition, CVPR 2017*, *2017-Janua*, 7120–7129. https://doi.org/10.1109/CVPR.2017.753

Annaswamy, A. M., & Fradkov, A. L. (2021). A Historical Perspective of Adaptive Control and Learning. *Annual Reviews in Control*, *52*, 18–41. https://doi.org/10.1016/j.arcontrol.2021.10.014

Bhagat, S., Banerjee, H., Tse, Z. T. H., & Ren, H. (2019, January). Deep reinforcement learning for soft, flexible robots: Brief review with impending challenges. https://doi.org/10.3390/robotics8010004
Meta-studie.

Björck, Å. (1996, January). *Numerical Methods for Least Squares Problems*. Society for Industrial; Applied Mathematics. https://doi.org/10.1137/1.9781611971484

Brochu, E., Cora, V. M., & de Freitas, N. (2010). A Tutorial on Bayesian Optimization of Expensive Cost Functions, with Application to Active User Modeling and Hierarchical Reinforcement Learning. *arXiv*, *abs/1012.2*. http://arxiv.org/pdf/1012.2599.pdf%20http://arxiv.org/abs/1012.2599

Chatzilygeroudis, K., Cully, A., & Mouret, J.-B. (2016). Towards semi-episodic learning for robot damage recovery. http://arxiv.org/abs/1610.01407

Cully, A., Clune, J., Tarapore, D., & Mouret, J. B. (2015). Robots that can adapt like animals. *Nature*, *521*(7553), 503–507. https://doi.org/10.1038/nature14422

Fujimoto, S., Van Hoof, H., & Meger, D. (2018). Addressing Function Approximation Error in Actor-Critic Methods. *35th International Conference on Machine Learning, ICML 2018*, *4*, 2587–2601. https://arxiv.org/abs/1802.09477v3

Gijsberts, A., & Metta, G. (2011). Incremental learning of robot dynamics using random features. *Proceedings - IEEE International Conference on Robotics and Automation*, 951–956. https://doi.org/10.1109/ICRA.2011.5980191

Gillespie, M. T., Best, C. M., Townsend, E. C., Wingate, D., & Killpack, M. D. (2018). Learning nonlinear dynamic models of soft robots for model predictive control with neural networks. *2018 IEEE International Conference on Soft Robotics (RoboSoft)*, 39–45. https://doi.org/10.1109/ROBOSOFT.2018.8404894

Giorelli, M., Renda, F., Calisti, M., Arienti, A., Ferri, G., & Laschi, C. (2015). Learning the inverse kinetics of an octopus-like manipulator in three-dimensional space. *Bioinspiration & Biomimetics*, *10*(3), 035006. https://doi.org/10.1088/1748-3190/10/3/035006

Haarnoja, T., Zhou, A., Abbeel, P., & Levine, S. (2018). Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. *35th International Con-*

ference on Machine Learning, ICML 2018, *5*, 2976–2989. https://arxiv.org/abs/1801.01290v2

Han, D., Mulyana, B., Stankovic, V., & Cheng, S. (2023, April). A Survey on Deep Reinforcement Learning Algorithms for Robotic Manipulation. https://doi.org/10.3390/s23073762

Hartmann, F., Baumgartner, M., & Kaltenbrunner, M. (2021, May). Becoming Sustainable, The New Frontier in Soft Robotics. https://doi.org/10.1002/adma.202004413

Hawkes, E. W., Blumenschein, L. H., Greer, J. D., & Okamura, A. M. (2017). A soft robot that navigates its environment through growth. *Science Robotics*, *2*(8), 1–8. https://doi.org/10.1126/scirobotics.aan3028

Hinton, G., Vinyals, O., & Dean, J. (2015). Distilling the Knowledge in a Neural Network. https://arxiv.org/abs/1503.02531v1%20http://arxiv.org/abs/1503.02531

Ibarz, J., Tan, J., Finn, C., Kalakrishnan, M., Pastor, P., & Levine, S. (2021). How to train your robot with deep reinforcement learning: lessons we have learned. *International Journal of Robotics Research*, *40*(4-5), 698–721. https://doi.org/10.1177/0278364920987859

Kim, D., Kim, S.-H., Kim, T., Kang, B. B., Lee, M., Park, W., Ku, S., Kim, D., Kwon, J., Lee, H., Bae, J., Park, Y.-L., Cho, K.-J., & Jo, S. (2021). Review of machine learning methods in soft robotics (G. Gu, Ed.). *PLOS ONE*, *16*(2), e0246102. https://doi.org/10.1371/journal.pone.0246102

Kullback, S., & Leibler, R. A. (1951). On Information and Sufficiency. *The Annals of Mathematical Statistics*, *22*(1), 79–86. https://doi.org/10.1214/aoms/1177729694

Landau, I. D., Lozano, R., M'Saad, M., & Karimi, A. (2011). *Adaptive Control*. Springer London. https://doi.org/10.1007/978-0-85729-664-1

Lesort, T., Lomonaco, V., Stoian, A., Maltoni, D., Filliat, D., & Díaz-Rodríguez, N. (2020). Continual learning for robotics: Definition, framework, learning strategies, opportunities and challenges. *Information Fusion*, *58*, 52–68. https://doi.org/10.1016/j.inffus.2019.12.004

Levine, S., & Koltun, V. (2013). Guided policy search. *30th International Conference on Machine Learning, ICML 2013*, *28*(PART 2), 1038–1046. https://proceedings.mlr.press/v28/levine13.html

Li, Z., & Hoiem, D. (2018). Learning without Forgetting. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, *40*(12), 2935–2947. https://doi.org/10.1109/TPAMI.2017.2773081

Lu, K., Grover, A., Abbeel, P., & Mordatch, I. (2020). Reset-Free Lifelong Learning with Skill-Space Planning. *ICLR 2021 - 9th International Conference on Learning Representations*. http://arxiv.org/abs/2012.03548

Lu, K., Mordatch, I., & Abbeel, P. (2019). Adaptive Online Planning for Continual Lifelong Learning. https://doi.org/10.48550/arXiv.1912.01188

Mouret, J.-B., & Clune, J. (2015). Illuminating search spaces by mapping elites. http://arxiv.org/abs/1504.04909

Nazeer, M. S., Bianchi, D., Campinoti, G., Laschi, C., & Falotico, E. (2023). Policy Adaptation using an Online Regressing Network in a Soft Robotic Arm. *2023 IEEE International Conference on Soft Robotics, RoboSoft 2023*, 1–7. https://doi.org/10.1109/RoboSoft55895.2023.10121927

Nguyen-Tuong, D., & Peters, J. (2011a). Incremental online sparsification for model learning in real-time robot control. *Neurocomputing*, *74*(11), 1859–1867. https://doi.org/10.1016/j.neucom.2010.06.033

Nguyen-Tuong, D., & Peters, J. (2011b). Model learning for robot control: a survey. *Cognitive Processing*, *12*(4), 319–340. https://doi.org/10.1007/s10339-011-0404-1

Nguyen-Tuong, D., Seeger, M., & Peters, J. (2008). Local Gaussian Process Regression for Real Time Online Model Learning and Control. *NIPS'08: Proceedings of the 21st International Conference on Neural Information Processing Systems*, 1193–1200. https://doi.org/10.5555/2981780.2981929

Nguyen-Tuong, D., Seeger, M., & Peters, J. (2009). Model Learning with Local Gaussian Process Regression. *Advanced Robotics*, *23*(15), 2015–2034. https://doi.org/10.1163/016918609X12529286896877

Pitman, J. (2006). Combinatorial stochastic processes. *Lecture Notes in Mathematics*, *1875*, 1–247. https://doi.org/10.1007/b11601500

Romeres, D., Zorzi, M., Camoriano, R., & Chiuso, A. (2016). Online semi-parametric learning for inverse dynamics modeling. http://arxiv.org/abs/1603.05412

Sayed, A. H. (2008, March). *Adaptive Filters*. Wiley. https://doi.org/10.1002/9780470374122

Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal Policy Optimization Algorithms. http://arxiv.org/abs/1707.06347

Schwarz, J., Luketina, J., Czarnecki, W. M., Grabska-Barwinska, A., Teh, Y. W., Pascanu, R., & Hadsell, R. (2018). Progress & compress: A scalable framework for continual learning. *35th International Conference on Machine Learning, ICML 2018*, *10*, 7199–7208. http://arxiv.org/abs/1805.06370

Shin, H., Lee, J. K., Kim, J., & Kim, J. (2017). Continual Learning with Deep Generative Replay. http://arxiv.org/abs/1705.08690

Sprechmann, P., Jayakumar, S. M., Rae, J. W., Pritzel, A., Badia, A. P., Uria, B., Vinyals, O., Hassabis, D., Pascanu, R., & Blundell, C. (2018). Memory-based parameter adaptation. *6th International Conference on Learning Representations, ICLR 2018 - Conference Track Proceedings*. http://arxiv.org/abs/1802.10542

Sun, Y., Abudula, A., Yang, H., Chiang, S.-S., Wan, Z., Ozel, S., Hall, R., Skorina, E., Luo, M., & Onal, C. D. (2021). Soft Mobile Robots: a Review of Soft Robotic Locomotion Modes. *Current Robotics Reports*, *2*(4), 371–397. https://doi.org/10.1007/s43154-021-00070-5

Tang, J., & Wei, F. (2022). Miniaturized Origami Robots: Actuation Approaches and Potential Applications. *Macromolecular Materials and Engineering*, *307*(2), 2100671. https://doi.org/10.1002/mame.202100671

Thuruthel, T. G., Falotico, E., Renda, F., & Laschi, C. (2019). Model-Based Reinforcement Learning for Closed-Loop Dynamic Control of Soft Robotic Manipulators. *IEEE Transactions on Robotics*, *35*(1), 124–134. https://doi.org/10.1109/TRO.2018.2878318

Wang, Z., Chen, C., & Dong, D. (2020). Lifelong Incremental Reinforcement Learning with Online Bayesian Inference. *IEEE Transactions on Neural Networks and Learning Systems*, *33*(8), 4003–4016. https://doi.org/10.1109/TNNLS.2021.3055499

Wang, Z., Novikov, A., Zolna, K., Springenberg, J. T., Reed, S., Shahriari, B., Siegel, N., Merel, J., Gulcehre, C., Heess, N., & de Freitas, N. (2020). Critic Regularized Regression. http://arxiv.org/abs/2006.15134

Williams, G., Aldrich, A., & Theodorou, E. (2015). Model Predictive Path Integral Control using Covariance Variable Importance Sampling. https://arxiv.org/abs/1509.01149v3%20http://arxiv.org/abs/1509.01149

Xie, A., & Finn, C. (2021). Lifelong Robotic Reinforcement Learning by Retaining Experiences. *Proceedings of Machine Learning Research*, *199*, 838–855. http://arxiv.org/abs/2109.09180

Yang, Y., Vella, K., & Holmes, D. P. (2021). Grasping with kirigami shells. *Science Robotics*, *6*(54). https://doi.org/10.1126/scirobotics.abd6426

Ze, Q., Wu, S., Nishikawa, J., Dai, J., Sun, Y., Leanza, S., Zemelka, C., Novelino, L. S., Paulino, G. H., & Zhao, R. R. (2022). Soft robotic origami crawler. *Science Advances*, *8*(13), 7834. https://doi.org/10.1126/sciadv.abm7834

Zhang, T., & Mo, H. (2021). Reinforcement learning for robot research: A comprehensive review and open issues. *International Journal of Advanced Robotic Systems*, *18*(3), 172988142110073. https://doi.org/10.1177/17298814211007305

Zhou, W., Bohez, S., Humplik, J., Abdolmaleki, A., Rao, D., Wulfmeier, M., Haarnoja, T., & Heess, N. (2022). Forgetting and Imbalance in Robot Lifelong Learning with Off-policy Data. *Proceedings of Machine Learning Research*, *199*, 294–309. http://arxiv.org/abs/2204.05893